Modisoft — Inventory Optimization PRD (v1)

Owner: Ebuka (AI PM)

Teams: Frontend (Nishant + UI), Backend (Nishant)

Integrations: Modisoft Back Office, Demand Forecasting (DF), Sunny (Sales/Inventory Agent; *no action-taking in v1*)

v1.2 - Changed ML model from Prophet to LightGBM and added Alerting & Notifications (Section X) to give store owners an early "heads-up" when demand is about to jump so they can prep staff and stock, without spamming them or auto-changing anything

Scope mode: Counts-only + forecasts. Optimization recommendations & suggestions based on forecasts and users' available inventory with insights on how to improve inventory turnover rate to boost sales. No agentic writes yet.

1) Purpose

Give store owners a simple, reliable way to:

1. See what they **have** and what they **can sell** given current on-hand and near-term forecasts.
2. Spot what's **wrong** with inventory data (stale, negative, phantom) and suggest fixes.
3. Get **clear suggestions** to improve stock position, prevention of stock-outs, and reduce slow/old stock.

This module **shares data** with the DF module and exposes read endpoints so **Sunny** can pull the same insights in chat.

2) In-Scope (v1)

- Two core widgets: **What you have** and **What you can sell** with list/bar/pie views.
- Near-term forecast horizons: **6h, 1d, 3d** (from DF tables).
- Data health flags: **negative** on-hand, **stale** last count (>14d default).
- **AI Insights** widget: 2–3 short, actionable statements generated by LLM from backend facts.
- **Quick Cycle Count** view with **Suggested On-Hand (SOH)** value per item (deterministic + light ML later) and reason.
- **Auto-Replenishment (PO Draft math only)**: calculates order quantities using simple target-days rule and case rounding; delivers numbers to UI and PO service **without** creating POs automatically.
- Shared data/logic endpoints for Sunny chat to surface the same results.

Out-of-Scope (v1)

- No vendor **lead times/ETAs**.
- No auto-editing inventory. No auto-creating POs. No transfers/adjustments.
- No computer vision shelf detection.
- No price optimization or promotion engine.

3) Users & Jobs-To-Be-Done

Primary: SMB owners/managers (c-store, grocery, liquor, restaurant).

- *"Tell me what's running out soon and what I should do."*
- *"Show me if my counts are wrong and what the count should likely be."*
- *"Size my next order roughly so I don't stock out."*

Secondary: Back-office staff; Ops/Area managers using Sunny chat.

Success depends on: simple visuals, plain language, zero "black-box" math.

4) Core Concepts & Rules

- **Forecast horizon** (F_h): provided by DF as 6h/1d/3d.
- **Sellable** = `min(OnHand, F_h)`.
- **Shelf-out likely** (display badge) if `OnHand > 0 && F_h > OnHand && recentSalesHours ≤ 2`.
- **Data health**: `OnHand < 0 → suspect`, `lastCountAge > 14d → stale`.
- **Suggested On-Hand (SOH)** per item (used in Cycle Count):
  - Primary estimate: `SOH ≈ max(0, LastPurchaseQty − DaysSincePurchase × DailyPace)`; fallback `DailyPace`.
  - `DailyPace` initially = DF `f1d` proxy; Phase-2 uses smoothed pace (Kalman/ETS).
- **Auto-Replenishment math (no write):**
  - `OrderQty = ceil_to_case( max(0, targetDays × DailyPace − (OnHand + OnOrder)) )`
  - Default `targetDays` by business type (can be edited later):
    - Convenience **3d**, Grocery **4d**, Liquor **7d**, Restaurant **2d**.

Date ranges: Align with DF to reduce cognitive load. **Use 7d, 14d, 28d** history presets for filters and analytics (trend panes), while **operational horizons** remain 6h/1d/3d. This keeps the same mental model as DF.

5) LLM & ML Usage (v1)

- **LLM:** GPT-4.1-mini for **AI Insights text only**. Input is a JSON blob of computed facts. Output is 2–3 concise suggestions with a short "why". No numbers invented; no tool calls that mutate data.
- **Rules first:** all sellable math, data-health, and Auto-PO are deterministic.
- **Light ML: Pace smoothing**: Kalman/ETS per item-store; stored as `daily_pace_smoothed`.
  - **Anomaly tagging** (later): robust z-score or Isolation Forest for phantom/shrink suspicion (read-only tag).

6) UX Requirements

6.1 Dashboard

- **Header controls:** Business type, store multi-select, dept filter, horizon toggle (6h/1d/3d), search.
- **KPIs:** Items at risk, Money tied in slow movers, Stock-outs prevented, Data-health fixes; Restaurant shows Waste risk.
- **What you have:** list/bar/pie; shows OH, On-order, last count, status chip (Good/Stale/Suspect), micro bullet bar with forecast marker and coverage pill.
- **What you can sell:** list/bar/pie; shows Forecast, Sellable, On-hand, shelf-out badge.
- **AI Insights** (always visible): 2–3 items with short "why".
- **Quick Cycle Count:** table of stale/suspect items with SOH suggestion + reason line (`last purchase`, `pace`, `days`).
- **Auto-Replenishment:** table for top items with computed order qty (case rounded). Button "Create PO" just navigates to Purchasing service with a prefilled cart payload (no write here).

6.2 Sunny Chat (read-only)

- Sunny can answer: "What should I fix in beverages at Store 101 today?"
- Sunny calls our read endpoint(s) and returns the same AI Insights in text form. No adjustments or PO creation.

X) Alerting & Notifications (Inventory Optimization) – v1

Goal

Use the existing Inventory Optimization logic (sellable, data-health flags, suggested OH, auto-PO math) to proactively warn the user when:

1. They're about to **run out** of key items.
2. They are **sitting on too much slow stock**.
3. Their **inventory data is trash** (negative / very stale counts).

All alerts are **read-only**. They never change inventory or create POs. They just point the user to the IO dashboard / PO screen.

X.1 In-Scope (v1)

1. **Daily Inventory Health email** per tenant / store group:
    a. Summary of:
        i. Items at **stock-out risk** (next 24–48h).
        ii. Money tied in **slow movers**.
        iii. Number of **data-health fixes** needed (negative / stale counts).
        iv. For restaurants: a short **waste risk** section.
2. **Push notification to Modisoft mPOS** when:
    a. Stock-out risk for key items crosses threshold **today / next 24h**.
3. **Rules-based engine** re-using IO and DF data:
    a. Uses **sellable = min(OnHand, Forecast)**, data-health flags, and simple pace metrics already defined in the PRD.
    b. No new ML models just for alerts.
4. **Alert log / API**:
    a. Store alerts in an `alerts_inv` table.
    b. Expose `GET /invopt/alerts?store_id=&date=` so IO dashboard and Sunny can show “Inventory alerts”.

X.2 Out-of-Scope (v1)

- No auto-creating POs or auto-editing inventory.
- No vendor ETA / lead-time logic.
- No per-item alert configuration in the UI (global thresholds only).
- No LLM-generated emails (plain templates, we only fill numbers).

X.3 Alert Types & Rules

All thresholds go into config so Ganesh can tune them without redeploy.

A. Stock-out risk (next 24–48h)

Purpose: warn "these items are going to zero if you don't act."

For each `{store, item}`:

1. Choose horizon:
    a. If store is open now and horizon on dashboard is 6h, use `f6h`.
    b. For daily email, use **`f1d` or `f3d`** depending on config.
2. Compute coverage:

```
coverage = OnHand / max(1, Forecast_h)
```

3. Stock-out risk if:

- `Forecast_h > 0`, and
- `coverage < STOCKOUT_COV_THRESHOLD` (default: **0.6**), and
- Item is active: `recentSalesHours <= ACTIVE_HOURS` (default: **24h**).

We tag risk levels:

- `coverage < 0.3` → **High**
- `0.3 ≤ coverage < 0.6` → **Medium**

The alert payload includes:

- Top N items by **revenue** or **units**.
- For each: `name, upc, on_hand, forecast_h, sellable, coverage, quality`.

These are the same fields you already define in the IO Item Row model.

B. Slow-mover / overstock dollars

Purpose: show "cash stuck in stuff that doesn't move."

Per `{store, item}`:

1. Approx daily pace: `daily_pace = max(0, f1d)` or `daily_pace_smoothed` when ML flag is on.
2. Compute days of stock:

```
days_of_stock = OnHand / max(1, daily_pace)
```

3. Slow mover if:

- `days_of_stock > SLOW_DAYS_THRESHOLD` (default: **45 days**), and
- Item not flagged as stock-out risk, and
- `OnHand * unit_cost >= SLOW_DOLLAR_MIN` (config; e.g. **$200**).

Email shows:

- Total **$ in slow movers** (ties to "Money tied in slow movers" KPI).
- Top 5–10 items by dollars.

C. Data-health issues

Purpose: call out "your inventory data is wrong, fix this first."

Use existing IO rules:

- Negative OnHand → **suspect**.
- `lastCountAge > 14d` → **stale**.

Alert triggers if, per store:

- `negative_count_items >= NEG_COUNT_THRESHOLD` (e.g. **> 5 items**), or
- `stale_count_items >= STALE_COUNT_THRESHOLD` (e.g. **> 20 items**).

We include:

- Total **data-health fixes** (matches dashboard KPI).
- 5–10 worst offenders for Quick Cycle Count (these map directly to rows in the Cycle Count widget).

D. Restaurant waste risk (restaurants only)

For `businessType = restaurant`:

- For prepared items with shelf-life metadata:
  - `hours_left <= WASTE_HOURS_THRESHOLD` (e.g. **≤ 24h**) and
  - `stockSell` (what you can sell before expiry) > 0.

Summarize per store:

- `count_at_risk`, `estimated_cost` (if unit cost available), top 3 items.

X.4 Frequency & Channels

1. **Daily Inventory Health email**
- Run after IO/DF refresh (e.g. **07:00 local time**).
- One email per tenant / store group.
- Covers horizon **today + next 2 days**.
2. **Same-day push notifications (mPOS)**
- Lightweight job every **60 min** during open hours:
    - Evaluate stock-out risk rule for top N items by revenue.
    - If high-risk items appear and **no push has been sent today**, send one notification.
- Cap: **1 IO push / store / day**.

X.5 Data Inputs

Alert engine reads from existing IO/DF structures:

- From `inventory_snapshot` / IO read model:
    - `on_hand`, `on_order`, `quality`, `last_count_ts`.
- From DF forecast tables:
    - `f6h`, `f1d`, `f3d`.
- From purchases:
    - `unit_cost` (optional; needed for dollar metrics).
- From optional feature table:
    - `daily_pace_smoothed` if enabled.

No external model APIs; everything runs inside the same IO / DF pipelines.

X.6 Backend Design (IO Alerts Service)

Scheduled job (Prefect / cron / Azure Function):

1. For each store:
    a. Read IO item rows for the relevant horizon (e.g. `f1d` or `f3d`).
    b. Compute `coverage`, `days_of_stock`, data-health aggregations.
2. Build an alert JSON per store:

```
{
  "storeId": "c101",
  "date": "2025-11-06",
  "inventory": {
    "stockoutRisk": {
      "count": 8,
      "items": [
        { "upc": "049000050103", "name": "Coca-Cola 20oz", "onHand":
18,
          "forecast1d": 52, "sellable": 18, "coverage": 0.35,
"quality": "good" }
      ]
    },
    "slowMovers": {
      "dollars": 2450,
      "items": [
        { "upc": "070847023501", "name": "Red Bull 12oz", "onHand":
120,
          "daysOfStock": 60, "dollars": 900 }
      ]
    },
    "dataHealth": {
      "negativeCount": 12,
      "staleCount": 45,
      "topOffenders": [ "Hot Dog Bun 8ct", "Gatorade 28oz" ]
    },
    "wasteRisk": {
      "count": 3,
      "dollars": 180
    }
  }
```

```
}
```

3. Persist to `alerts_inv`:
- `tenant_id, store_id, alert_date, type, severity, payload_json, channels_sent`.
4. Send:
- **Email** → internal email service (aggregated by tenant).
- **Push** → mPOS push API (stock-out risk only).

API:

- `GET /invopt/alerts?store_id=&date=`
    - Returns latest IO alerts payload.
    - Used by IO dashboard to show an "Alerts" panel and by Sunny to answer questions like
      "Anything scary in beverages today?"

X.7 Email Template (Inventory Health)

Subject options

- `Inventory health check for {StoreName} (today + 2 days)`
- `Heads up: stock risk & slow movers at {StoreName}`

Body skeleton

Hi {FirstName},

Here's a quick inventory snapshot for **{StoreName}**.

1. Items at stock-out risk (next {HorizonText})

- **{count_at_risk} items** are likely to run low.
- Top examples:
    - **{Item1}** – OH {oh1}, forecast {fcst1}, coverage {cov1}x
    - **{Item2}** – OH {oh2}, forecast {fcst2}, coverage {cov2}x
- Suggested action: **review these in Inventory Optimization → "What you can sell"**.

2. Slow movers / overstock

- About **${slow_dollars}** tied up in slow-moving items.
- Biggest offenders: **{SlowItem1}, {SlowItem2}**.
- Suggested action: consider **promo, discount, or smaller reorders**.

3. Data-health fixes

- **{negative_count} items** have **negative counts**.
- **{stale_count} items** haven't been counted in over **14 days**.
- Suggested action: run a **Quick Cycle Count** on these items.

{If restaurant:}

4. Waste risk (next 48h)

- **{waste_count} prepared items** may expire soon. Prioritize selling or adjust prep.

These are suggestions only. We don't change your inventory or orders automatically.

The backend simply fills the { } placeholders with computed metrics.

X.8 Push Notification Text (IO)

Examples (once per store per day):

- `Inventory alert: {count_at_risk} items may run out soon at {StoreName}. Tap to review.`
- `Heads up: Coca-Cola 20oz and 3 other items are at stock-out risk today.`

Tap → deep link to IO dashboard filtered to "Items at risk" for that store.

X.9 Acceptance Criteria (IO Alerts)

1. **Correct triggers**
    a. When `coverage < STOCKOUT_COV_THRESHOLD`, item appears under "stock-out risk" in `alerts_inv` and email.

   b. When `days_of_stock > SLOW_DAYS_THRESHOLD`, item contributes to slow-mover dollars.
   c. Negative / stale counts match IO data-health counts.
2. **No spam**
   a. At most **one** IO email per tenant per day.
   b. At most **one** IO push per store per day.
3. **Data correctness**
   a. Coverage and days-of-stock numbers in email match IO calculations (± rounding).
   b. Counts in email (items at risk, slow-mover dollars, data-health items) equal aggregates computed from IO rows.
4. **Parity**
   a. For the same date/store, IO dashboard "Alerts" panel shows the same items as the email.
   b. Sunny's "inventory alerts" answers use the same `alerts_inv` payload.
5. **Performance**
   a. Daily IO alert job finishes within **5 minutes** after IO/DF refresh.
   b. `GET /invopt/alerts` p95 latency ≤ **500 ms**.

7) Data Contracts

7.1 Item Row (read model)

```
{
  "upc": "number",
  "name": "string",
  "dept": "string",
  "vendor": "string",
  "abc": "A|B|C",
  "on_hand": number,
  "on_order": number,
  "f6h": number,
  "f1d": number,
  "f3d": number,
  "horizon": "6h|1d|3d",
  "Fh": number,               // selected horizon forecast
```

```
  "sellable": number,         // min(on_hand, Fh)
  "recent_sales_hours": number,
  "last_count_ts": "ISO",
  "last_purchase_qty": number|null,
  "last_purchase_ts": "ISO|null",
  "quality": "good|stale|suspect", // computed from rules
  "suggested_oh": { "value": number, "ci_low": number|null, "ci_high":
number|null, "confidence": number|null }
}
```

7.2 AI Insights (LLM output schema)

```
{
  "insights": [
    {
      "title": "string (≤80)",
      "detail": "string (≤240)",
      "reason": { "upc": "string", "oh": number, "forecast": number,
"sellable": number, "last_count_age_days": number },
      "cta": { "type": "open_report"|"open_cycle_count"|"open_po",
"payload": {"filter": {...}} }
    }
  ]
}
```

8) Backend APIs (Inventory Optimization Service)

All GETs are read-only; POST for LLM explain only. No writes to inventory or PO in v1.

GET `/invopt/rows`

Params: `store_ids[]`, `dept`, `horizon in {6h,1d,3d}`, `limit`

Returns: list of **Item Row** objects.

GET `/invopt/health`

Params: `store_ids[]`, `dept`

Returns: counts of `stale`, `suspect`, top offenders (for Cycle Count view).

GET `/invopt/auto-po`

Params: `store_ids[]`, `dept`, `target_days`, `case_pack_strategy=default|vendor_override`

Returns: `{ lines:[{ upc, name, on_hand, on_order, daily_pace, target, order_qty }], summary:{items,n_units} }`

POST `/invopt/ai/insights`

Body: `{ rows:[ItemRow], context:{businessType,horizon,stores,dept} }`

Returns: **AI Insights** schema. Uses GPT-4.1-mini; cached 5–15 min per filter state.

Security: RBAC on store scope; audit log request/response hashes.

9) Data Sources & Refresh

- **SQL views** for `sales_line`, `inventory_snapshot`, `purchase_receipts`/`po_line`, `df_forecast`.
- Refresh cadence: snapshots hourly; sales near-real-time; forecasts by DF schedule; purchases on receipt.
- Feature table (optional): `daily_pace_smoothed` (if ML flag on).

10) Acceptance Criteria (QA-testable)

1. **Sellable math**: Given `OH=10, F1d=12` → `Sellable=10` (cap).
2. **Shelf-out badge**: If `OH=6, F6h=9, recentSalesHours=1` → badge visible.
3. **Data health**: Negative OH shows `suspect`; last count older than 14d shows `stale`.
4. **SOH suggestion**:
    a. With last purchase `120` 6 days ago, `DailyPace=18` → SOH around `12` ± band.
    b. Without purchase history → SOH equals `f1d` proxy.

5. **Auto-PO**: With `DailyPace=20, targetDays=3, OH=10, OnOrder=0, case=24` → `Order=72`.
6. **AI Insights**: Always returns 2–3 items; each includes a one-sentence "why" referencing provided facts; never invents numbers.
7. **Sunny parity**: For the same filters, Sunny's chat answer matches dashboard insight ordering and numbers.
8. **Latency**: `/invopt/rows` p95 ≤ 1.2s; `/invopt/ai/insights` p95 ≤ 1.8s with cache.

11) Telemetry & Metrics

- Stock-out rate (item-hours at zero OH) — target ↓ 20%.
- Fill rate — target ↑ 5–10%.
- Slow-mover dollars — target ↓ 10–15%.
- Data-health incidents (stale/negative) — target ↓ 50% after 30 days.
- PO creation time & errors — target ↓ 30%.
- Dashboard → Sunny handoff rate & insight click-through.

12) Risks & Mitigations

- **Dirty counts** → keep flags + SOH suggestions with reasons.
- **Over-trust of AI** → LLM can't compute or write; shows sources/why.
- **Performance** → cache per `{stores,dept,horizon}`; limit rows for charts; summarize in AI.
- **Schema drift with DF** → formal data contract; versioned views.

13) Rollout Plan

- **Week 1–2**: API scaffolding (`/invopt/rows`, `/health`, `/auto-po`).
- **Week 3**: Dashboard v1 (have/sell widgets; cycle count; auto-po).
- **Week 4**: LLM insights endpoint + caching; Sunny read integration.

- **Week 5**: QA, seed data, telemetry dashboards; pilot in 2–3 stores.

14) Open Questions

1. Confirm **default targetDays** per business type (3/4/7/2).
2. Confirm **stale threshold** remains 14 days.
3. Case pack source of truth: item master only, or vendor overrides?
4. Any legal/UX copy edits for AI phrasing?

15) Appendix — Pseudocode

f1d = "forecast for 1 day."
It's the number of units our Demand Forecasting module predicts you'll sell in the next **1 day** for a given item/store.

Quick map:

- **f6h** = forecast for next **6 hours**
- **f1d** = forecast for next **1 day**
- **f3d** = forecast for next **3 days**

Where we use it:

- **Sellable math:** sellable = min(On-Hand, Fh) where Fh is whichever horizon you picked (so if you chose 1d, Fh = f1d).
- **Shelf-out flag:** compares On-Hand vs the chosen forecast (e.g., f1d).
- **Suggested OH fallback:** if we don't have a good last-purchase signal, we temporarily use f1d as the **daily pace** proxy.

```
sellable = Math.max(0, Math.min(on_hand, Fh));
quality  = on_hand < 0 ? 'suspect' : isStale(last_count_ts, 14) ?
'stale' : 'good';
shelfOut = on_hand > 0 && Fh > on_hand && recent_sales_hours <= 2;
```

```
// Suggested On-Hand (primary)
if (last_purchase_qty && last_purchase_ts) {
  days = daysSince(last_purchase_ts)
  SOH = max(0, round(last_purchase_qty - days * daily_pace))
} else {
  SOH = round(daily_pace) // fallback
}

// Auto-PO
need = targetDays * daily_pace - (on_hand + on_order)
order = ceilToCase(max(0, round(need)), casePack)
```